# AI Hiring Assistant Backend

Project explained in simple terms: components, workflow and architecture

> **The resume line**
> "Delivered sub-second latency for 1K+ daily users by designing distributed Python/FastAPI backend services with well-defined interfaces orchestrating LLM inference, Greenhouse ATS, and calendar APIs."

## 1. The Big Picture (Simple Summary)

Hiring people involves a lot of small, boring tasks: looking up a candidate, checking who is free for an interview, booking the meeting, and updating the hiring system. Recruiters normally do this by jumping between three or four different tools.

This project built the **backend (the behind-the-scenes engine)** of an **AI hiring assistant**. A recruiter can simply type a request in plain English, such as "Book a 45-minute interview for Priya with Alex and Sam next week". The system then:

1. uses **AI (an LLM)** to understand what the person wants,
2. looks up the candidate in **Greenhouse** (the company's hiring database, called an ATS),
3. checks the interviewers' **calendars** to find a time when everyone is free,
4. and replies with the best options, then books the meeting and updates Greenhouse.

Instead of one big program, the backend is split into several **small services**, each with one clear job (one for AI, one for Greenhouse, one for calendars, and one "orchestrator" that coordinates them). Each service has a **well-defined interface**, meaning a clear, agreed set of requests it accepts and answers it returns, so the services can be built, changed and scaled separately.

The result: **more than 1,000 people use it every day, and they get answers in under one second.** That speed comes from four main tricks: doing work **in parallel**, **caching** repeated answers, pushing slow work to the **background**, and running **many copies** of each service.

**One-sentence version**

I built the fast, reliable engine behind an AI recruiting assistant: it understands plain-English requests, talks to Greenhouse and calendar systems at the same time, and answers 1,000+ daily users in under a second.

## 1.1 The resume line, decoded word by word

| Phrase | What it means in simple terms |
|---|---|
| Delivered sub-second latency | Each request gets a reply in less than 1 second (measured at p95, meaning 95 out of every 100 requests). |
| for 1K+ daily users | More than 1,000 different people use it on a typical day. |
| designing distributed backend services | The system is split into several small programs (services) that run separately and talk over the network, instead of one big program. |
| Python/FastAPI | Python is the programming language. FastAPI is a modern Python web framework that is quick and supports async (doing many things at once without waiting). |
| with well-defined interfaces | Every service has a clear contract: exactly what inputs it accepts and what outputs it returns (for example, documented REST endpoints with typed Pydantic models). |
| orchestrating | Coordinating several services, deciding who to call, in what order, and combining their answers. |
| LLM inference | Asking an AI language model (like GPT or Claude) to read text and produce an answer. |
| Greenhouse ATS | Greenhouse is a popular Applicant Tracking System, the software companies use to store candidates, jobs and interview stages. |
| calendar APIs | Google Calendar and Microsoft Outlook (Graph) programming interfaces used to check free time and create meetings. |

## 1.2 Key terms used in this document

| Term | Simple meaning |
|---|---|
| API | A menu of requests one program can make to another, and the answers it will get back. |

| Term | Simple meaning |
|---|---|
| Service / microservice | A small program that does one job and can be run and updated on its own. |
| Latency | How long a user waits for an answer. |
| p95 latency | The time within which 95% of requests finish. It shows what nearly every user experiences, not just the average. |
| Async / parallel | Starting several tasks at once and waiting for all of them together, instead of one after another. |
| Cache | A fast, short-term memory for answers that are needed again soon. |
| Queue | A waiting list of jobs that will be done shortly, in the background. |
| JWT | A signed digital ID card that proves who the user is. |
| Rate limit | A cap on how many requests are allowed in a time window. |
| Webhook | A message another system sends us automatically when something changes. |
| Idempotency | Doing the same request twice has the same effect as doing it once (no double bookings). |
| Stateless | A service that keeps no memory between requests, so any copy can handle any request. |

## 2. Architecture Diagram

The picture below shows every part of the system and how they connect. Read it from **top to bottom**: a request starts with the user, passes through the front door (gateway), reaches the orchestrator, fans out to the specialist services, and those services talk to outside systems at the bottom. Numbered circles match the table on the next page.

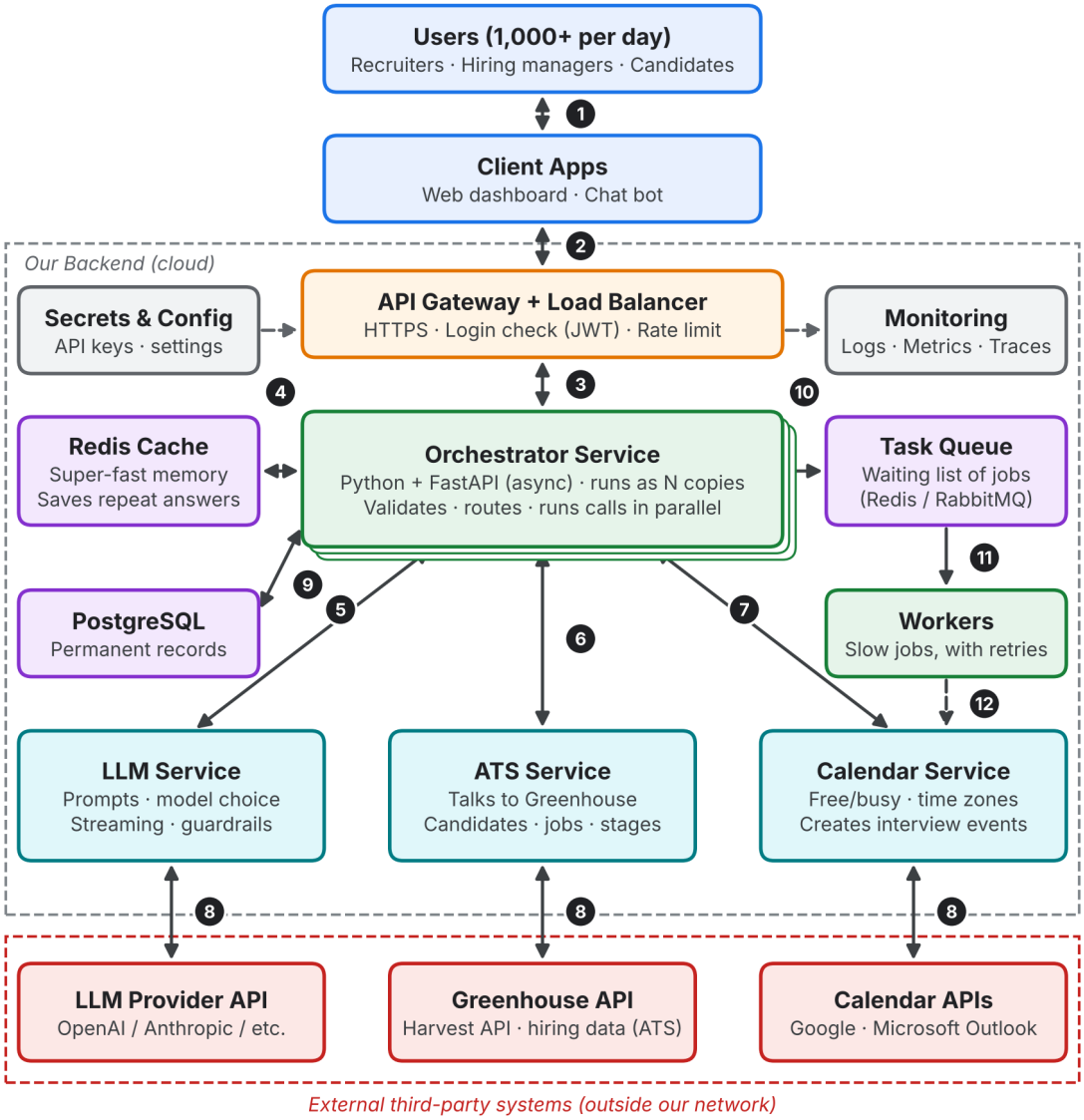



*Figure 1. System architecture of the AI hiring assistant backend*

## 2.1 How to read the diagram: colours and boxes

| Colour / shape | Meaning |
|---|---|
| Blue boxes | The people and the apps they use (outside the backend). |
| Orange box | The front door: API Gateway and Load Balancer. |
| Green boxes | Python code that does work: the Orchestrator and Background Workers. |
| Stacked green box | The Orchestrator runs as many identical copies (N copies) to handle many users at once. |
| Teal boxes | Specialist services, one per outside system (LLM, ATS, Calendar). |
| Purple boxes | Places where data is kept: Redis cache, PostgreSQL database, Task Queue. |
| Grey boxes | Supporting tools: Secrets & Config, Monitoring. |
| Red boxes | Outside companies' systems that we do not control. |
| Grey dashed frame | Everything inside belongs to our backend in the cloud. |
| Red dashed frame | Third-party systems outside our network. |
| Solid two-headed arrow | A live request and its reply (the user is waiting). |
| One-headed arrow | A one-way hand-off (no reply needed straight away). |
| Dashed arrow | Background or supporting traffic. |

## 2.2 Every numbered connection explained

| # | Connection | What travels along it, in simple terms |
|---|---|---|
| 1 | Users ↔ Client Apps | People use the web dashboard or chat bot to make requests and see answers. |
| 2 | Client Apps ↔ API Gateway | Encrypted HTTPS request with a login token goes in; the reply comes back the same way. |
| 3 | API Gateway ↔ Orchestrator | Checked, rate-limited requests are spread across the many Orchestrator copies. |
| 4 | Orchestrator ↔ Redis Cache | Quick look-ups of recent answers and reference data (about 1 ms). |

| # | Connection | What travels along it, in simple terms |
|---|---|---|
| 5 | Orchestrator ↔ LLM Service | Ask the AI to understand the request or write text; get structured results back. |
| 6 | Orchestrator ↔ ATS Service | Read or update candidate, job and stage information. |
| 7 | Orchestrator ↔ Calendar Service | Find free time and create interview events. Steps 6 and 7 run at the same time. |
| 8 | Each service ↔ its external API | Each internal service is the only place that talks to its outside provider: LLM provider, Greenhouse, or Google/Microsoft calendars. This line crosses the company network boundary. |
| 9 | Orchestrator ↔ PostgreSQL | Save permanent records (requests, bookings, job status, idempotency keys). |
| 10 | Orchestrator → Task Queue | One-way: drop slow jobs on the waiting list instead of making the user wait. |
| 11 | Task Queue → Workers | Workers pull jobs off the list one at a time. |
| 12 | Workers → Services (dashed) | Workers use the same ATS, Calendar and LLM services to finish the slow jobs (dashed means it happens in the background). |
| Grey dashed arrows | Secrets → Gateway, Gateway → Monitoring | Secrets and settings feed every service; every service sends logs, metrics and traces to Monitoring. Drawn once at the top to keep the picture clean. |

## 2.3 The diagram in layers

- **Layer 1, People and apps (top):** users and the screens they use.
- **Layer 2, Front door:** the API Gateway checks every request and shares the load.
- **Layer 3, Brain:** the Orchestrator decides what to do, with the cache, database and queue beside it.
- **Layer 4, Specialists:** the LLM, ATS and Calendar services, each an expert in one outside system.
- **Layer 5, Outside world (bottom):** the AI provider, Greenhouse and calendar providers.
- **Side helpers:** Secrets & Config on the left feeds settings to everything; Monitoring on the right watches everything.

Why this shape? Each outside system has exactly **one** service that talks to it. If Greenhouse changes its API or the company switches AI providers, only one small box changes. That is what "well-defined interfaces" buys you.

# 3. Every Component Explained

Each component below is described in the same way: what it is, a real-life comparison, exactly what it does step by step, the technology used, and how it helps keep responses under one second.

## 3.1 Users (1,000+ every day)

**In simple words:** The real people who use the product: recruiters, hiring managers, interviewers and sometimes candidates. More than a thousand of them use it on a normal day.

**Real-life comparison:** Customers walking into a restaurant.

**What it does, step by step:**

- A recruiter asks for something in plain English, such as "Schedule a 45-minute interview for Priya with Alex and Sam next week".
- A hiring manager might ask "Summarise the feedback for this candidate".
- A candidate might pick an interview time from a link they were sent.

**Technology:** No technology here; these are the people the whole system serves.

**How it helps speed:** 1,000+ users means many requests can arrive at the same moment. Every other part of the design exists so that each of them still gets an answer in under one second.

## 3.2 Client Apps (web dashboard and chat bot)

**In simple words:** The screens people actually touch: a web dashboard in the browser and a chat-style assistant (for example inside Slack or the web app).

**Real-life comparison:** The restaurant menu and the waiter's notepad. It collects the order but does not cook anything.

**What it does, step by step:**

- Shows a text box or buttons so the user can make a request.
- Attaches the user's login token (a JWT, a digital ID card) to every request.
- Sends the request over HTTPS (an encrypted connection) to the backend.
- Shows the answer, and can display it word by word as it streams in, so the user sees something almost instantly.

**Technology:** Any web or chat front end. It talks to the backend only through the published API, so the front end and backend can change independently.

**How it helps speed:** Streaming partial results means the user sees a reply starting in a fraction of a second, even while the rest is still being prepared.

## 3.3 API Gateway and Load Balancer

**In simple words:** The single front door to the backend. Every request comes in here first, gets checked, and is then handed to whichever backend copy is least busy.

**Real-life comparison:** A security guard at the door who checks your ID, plus a host who seats you at the shortest queue.

**What it does, step by step:**

- **Encryption (TLS):** unlocks the encrypted HTTPS message.
- **Authentication:** checks the JWT login token is real and not expired. Bad tokens are rejected immediately, before costing any real work.
- **Rate limiting:** stops any one user or script from sending too many requests (for example 60 per minute), so one noisy user cannot slow everyone else down.
- **Request ID:** stamps a unique ID on the request so it can be traced through every service later.
- **Load balancing:** spreads requests evenly across the many copies of the Orchestrator.
- **Health checks:** stops sending traffic to any copy that is unhealthy.

**Technology:** Typically a cloud load balancer (such as AWS ALB) or NGINX / Kong / Envoy in front of the services.

**How it helps speed:** Adds only about 5 to 20 milliseconds, but protects the system from overload, which is the most common cause of slow responses.

## 3.4 Orchestrator Service (Python + FastAPI)

**In simple words:** The brain of the system. It understands what the user wants, decides which other services to call, calls them at the same time where possible, combines their answers and sends one clean reply back.

**Real-life comparison:** The head chef. It does not cook every dish itself; it tells each station what to make, all at once, and plates the final meal.

**What it does, step by step:**

- **Receives** the request through a FastAPI endpoint such as POST /v1/assistant/requests.
- **Validates** the input with Pydantic models (it checks that required fields exist and have the right type). Bad input gets a clear error straight away.
- **Checks the cache** (Redis) to see if this exact question was answered recently.
- **Asks the LLM Service** to turn the free-text request into a structured instruction (for example: action = schedule_interview, candidate = Priya, length = 45 minutes).
- **Fans out in parallel:** uses Python async (asyncio.gather) to call the ATS Service and the Calendar Service at the same time instead of one after the other.

- **Sets a time limit** (timeout) on every call, so one slow service cannot freeze the whole request.
- **Merges** the results (for example, finds time slots when everyone is free).
- **Hands slow jobs** (emails, writing back to Greenhouse) to the Task Queue so the user does not wait for them.
- **Replies** to the user and records what happened in the database and logs.

**Technology:** Python, FastAPI, Uvicorn/Gunicorn workers, async HTTP client (httpx), Pydantic. It is stateless (it keeps no memory between requests), so many identical copies run side by side.

**How it helps speed:** Calling services in parallel is the single biggest speed win: if the ATS takes 250 ms and the calendar takes 200 ms, the total is about 250 ms, not 450 ms.

## 3.5 LLM Service (the AI part)

**In simple words:** A small dedicated service that wraps all communication with the large language model (the AI). Other services never talk to the AI directly; they ask this service.

**Real-life comparison:** A translator who turns messy human sentences into a neat, filled-in form, and can also write polite replies.

**What it does, step by step:**

- **Prompt templates:** keeps tested, versioned instructions for each task (understand request, summarise feedback, draft email).
- **Model routing:** uses a small, fast model for simple tasks like understanding the request, and a larger model only when quality really matters.
- **Structured output:** asks the AI to answer in strict JSON that matches a schema, then validates it, so the rest of the code can trust the result.
- **Streaming:** passes words back as they are generated, so users see text immediately.
- **Guardrails:** removes or masks personal data where not needed, blocks unsafe output, and checks the answer makes sense.
- **Cost and speed controls:** limits the length of prompts and answers, caches repeated prompts, and counts tokens used.
- **Fallbacks:** if one AI provider is slow or down, it retries or switches to a backup model.

**Technology:** Python + FastAPI wrapper around an LLM provider API (for example OpenAI, Anthropic or a self-hosted model).

**How it helps speed:** The AI call is usually the slowest single step, so keeping prompts short, using a fast model for routing, and streaming are what keep the total under one second.

## 3.6 ATS Service (Greenhouse integration)

**In simple words:** The service that talks to Greenhouse, the Applicant Tracking System (ATS) where the company keeps all its candidates, jobs and hiring stages.

**Real-life comparison:** A clerk who knows exactly how to find and update files in the HR filing cabinet.

**What it does, step by step:**

- **Authenticates** to the Greenhouse Harvest API with a secret API key (kept in the Secrets store, never in code).
- **Reads** candidates, applications, jobs, interview stages and scorecards.
- **Writes** updates such as scheduled interviews or notes. Greenhouse requires an "On-Behalf-Of" user ID on writes so every change is credited to a real person.
- **Respects rate limits:** Greenhouse only allows a limited number of calls every few seconds and reports this in response headers. The service tracks this and slows down before hitting the limit.
- **Handles pagination:** big lists come back in pages; the service fetches only the pages it needs.
- **Normalises data:** converts Greenhouse's format into the company's own simple, well-defined format so the Orchestrator never depends on Greenhouse details.
- **Caches** frequently read data (jobs, interview stages, user directory) in Redis for a few minutes.
- **Receives webhooks:** Greenhouse can notify the system when something changes (for example a candidate moves stage), which keeps the cache fresh.

**Technology:** Python + FastAPI, async HTTP client, retry with exponential backoff, Redis cache.

**How it helps speed:** Most reads are served from cache in 1 to 2 ms instead of 150 to 400 ms from Greenhouse.

## 3.7 Calendar Service (Google and Microsoft)

**In simple words:** The service that reads people's calendars to see when they are free, and creates interview meetings.

**Real-life comparison:** An assistant who looks at everyone's diaries and books a meeting room.

**What it does, step by step:**

- **Connects** with OAuth 2.0 (secure, permission-based login) to Google Calendar or Microsoft Graph (Outlook).
- **Free/busy lookup:** asks "when are Alex and Sam busy next week?" in one batch call rather than reading every event.

- **Time zones:** converts everything to UTC internally and back to each person's local time for display, so a 10:00 slot is 10:00 for the right person.
- **Slot finding:** works out the overlapping free windows that match the requested length and working hours.
- **Creates events** with attendees, a video link and a description.
- **Idempotency:** each booking carries a unique key, so if a request is retried, the meeting is not booked twice.
- **Refreshes tokens:** quietly renews expired OAuth access tokens.

**Technology:** Python + FastAPI, Google Calendar API, Microsoft Graph API, timezone library (zoneinfo).

**How it helps speed:** One batched free/busy call for all interviewers takes roughly 100 to 250 ms, and runs at the same time as the ATS lookup.

## 3.8 Redis Cache

**In simple words:** A super-fast, in-memory store for answers that are needed again and again.

**Real-life comparison:** A sticky note on your desk with phone numbers you use every day, so you do not have to open the phone book each time.

**What it does, step by step:**

- Stores recent ATS data (job lists, stage names, user directory).
- Stores results of identical AI requests (prompt caching) for a short time.
- Stores rate-limit counters and short-lived session data.
- Every item has an expiry time (TTL), for example 5 minutes, so data never gets too old.
- Items are removed early when a Greenhouse webhook says the data changed (cache invalidation).

**Technology:** Redis (often a managed service such as AWS ElastiCache).

**How it helps speed:** Reads take about 1 ms, compared with hundreds of milliseconds for an external API. A high cache hit rate is a big part of sub-second latency.

## 3.9 PostgreSQL Database

**In simple words:** The permanent record book. Things that must not be lost are saved here.

**Real-life comparison:** The company's official ledger.

**What it does, step by step:**

- Saves each request, its outcome and how long it took (audit trail).
- Saves scheduling records, idempotency keys and job status.
- Saves user settings and connected calendar accounts (with tokens encrypted).

- Uses indexes so look-ups stay fast as data grows, and a connection pool so services do not waste time opening new connections.

**Technology:** PostgreSQL with an async driver (asyncpg / SQLAlchemy) and a connection pool.

**How it helps speed:** Writes that the user does not need to wait for (such as logs) are done after the reply is sent, so they add no visible delay.

## 3.10 Task Queue

**In simple words:** A waiting list for jobs that are slow or can happen a few seconds later.

**Real-life comparison:** The ticket rail in a kitchen: orders are pinned up and cooks take them one by one.

**What it does, step by step:**

- The Orchestrator drops a small message on the queue, for example "write this interview to Greenhouse".
- The message stays safely in the queue until a worker picks it up, even if a service restarts.
- Failed jobs are put back for another try; jobs that keep failing go to a "dead-letter" queue for a human to inspect.

**Technology:** Redis Streams, RabbitMQ or AWS SQS, used through Celery, RQ or Arq.

**How it helps speed:** Moves slow work out of the user's path, so the user gets their answer without waiting for emails or third-party writes.

## 3.11 Background Workers

**In simple words:** Separate Python programs that pick jobs off the Task Queue and finish them.

**Real-life comparison:** Kitchen staff who handle the washing-up and deliveries while the chef keeps cooking.

**What it does, step by step:**

- Write scheduled interviews and notes back to Greenhouse.
- Send confirmation emails and calendar invites.
- Use the AI to prepare longer content, such as an interview kit or a feedback summary.
- Retry with exponential backoff (wait 1 s, then 2 s, then 4 s ...) if an external API is busy.
- Report success or failure so the user can see the final status.

**Technology:** Python workers (Celery / RQ / Arq) sharing code with the services.

**How it helps speed:** Workers can be scaled separately, so a burst of slow jobs never slows down live user requests.

## 3.12 Secrets and Configuration

**In simple words:** A locked safe for passwords and API keys, plus a settings file for the whole system.

**Real-life comparison:** The manager's key box.

**What it does, step by step:**

- Holds the Greenhouse API key, the LLM provider key and calendar OAuth secrets.
- Services read them at start-up; they are never written in the code.
- Holds settings such as timeouts, cache lifetimes and which AI model to use, so they can change without redeploying.

**Technology:** AWS Secrets Manager, HashiCorp Vault or Kubernetes Secrets, plus environment-based config.

**How it helps speed:** Indirect: tuning timeouts and cache settings here is how the team keeps latency on target.

## 3.13 Monitoring (logs, metrics, traces)

**In simple words:** The dashboard and alarms that show how healthy and fast the system is.

**Real-life comparison:** A car's dashboard, with speedometer and warning lights.

**What it does, step by step:**

- **Logs:** a written record of each step, tagged with the request ID.
- **Metrics:** numbers such as requests per second, error rate and p95 latency (the time within which 95 out of 100 requests finish).
- **Traces:** a timeline for one request showing how long each service took, so the slowest part is easy to spot.
- **Alerts:** notify engineers if p95 goes above 1 second or errors rise.

**Technology:** OpenTelemetry, Prometheus + Grafana, or Datadog; structured JSON logs.

**How it helps speed:** You cannot keep something under one second without measuring it. Traces showed where to add caching and parallel calls.

## 3.14 External APIs (outside the company)

**In simple words:** Systems run by other companies that this project depends on: the LLM provider, Greenhouse, and Google / Microsoft calendars.

**Real-life comparison:** Suppliers who deliver ingredients to the restaurant.

**What it does, step by step:**

- They can be slow, limit the number of calls, or occasionally fail.

- That is why each one is wrapped by its own service with timeouts, retries, caching and a simple, stable interface.
- If a provider changes its API, only one small service needs updating.

**Technology:** LLM provider API, Greenhouse Harvest API, Google Calendar API, Microsoft Graph API.

**How it helps speed:** Treating them as unreliable and slow by default (and protecting against it) is what makes the overall system fast and dependable.

## 3.15 How the services talk to each other (the interfaces)

"Well-defined interfaces" means every service publishes a clear, versioned contract. Callers only need to know the contract, not how the service works inside. Example contracts:

| Service | Example endpoint | Input | Output |
|---|---|---|---|
| Orchestrator | POST /v1/assistant/requests | user text, user ID | reply text, slot options, request ID |
| LLM Service | POST /v1/llm/extract-intent | text, task name | JSON: action, candidate, people, duration |
| ATS Service | GET /v1/ats/candidates/{id} | candidate ID | name, email, application, current stage |
| ATS Service | POST /v1/ats/interviews | application, time, interviewers | interview ID |
| Calendar Service | POST /v1/calendar/availability | emails, date range, length | list of free slots |
| Calendar Service | POST /v1/calendar/events | slot, attendees, idempotency key | event ID, video link |

- Requests and replies are JSON, with types checked by **Pydantic** on both sides.
- FastAPI generates **OpenAPI documentation** automatically, so every contract is written down and testable.
- Version numbers (**/v1/**) let a service change without breaking callers.
- Every call carries the **request ID** and has a **timeout**.

# 4. End-to-End Workflow

This section follows one real example from the moment a recruiter presses Send until everything is finished. Times are typical figures for this kind of design.

> **Example request**
> Recruiter types: **"Schedule a 45-minute technical interview for Priya Sharma with Alex and Sam next week."**

## 4.1 Part A: Finding interview times (user is waiting)

| # | Who | What happens, in simple terms | Time |
|---|---|---|---|
| 1 | User | Recruiter types: "Schedule a 45-minute technical interview for Priya Sharma with Alex and Sam next week" and presses Send. | 0 ms |
| 2 | Client App | Packages the text as JSON, attaches the login token (JWT) and sends it over HTTPS to POST /v1/assistant/requests. | ~20 ms network |
| 3 | API Gateway | Decrypts the request, verifies the token, checks the rate limit, adds request ID req-8f2a, and forwards it to the least-busy Orchestrator copy. | ~10 ms |
| 4 | Orchestrator | FastAPI receives it. Pydantic checks the body is valid. It loads the user's permissions (allowed to schedule for this job). | ~5 ms |
| 5 | Redis Cache | Orchestrator checks whether this exact request was answered in the last few minutes. Not found (a "cache miss"), so it continues. | ~1 ms |
| 6 | LLM Service | Turns the sentence into structured JSON: action = schedule_interview, candidate = "Priya Sharma", interviewers = [Alex, Sam], duration = 45, window = next week. A fast model and strict JSON schema are used; timeout is 600 ms. | ~300-450 ms |
| 7 | Orchestrator | Starts two calls at the same time with asyncio.gather (parallel fan-out). | 0 ms |

| # | Who | What happens, in simple terms | Time |
|---|---|---|---|
| 7a | ATS Service | Finds Priya's application and the "Technical Interview" stage. Job and stage data come from Redis; the candidate record comes from the Greenhouse Harvest API. Interviewer emails come from the cached user directory. | ~150-250 ms |
| 7b | Calendar Service | Makes one batched free/busy call for Alex and Sam for next week, converts all times to UTC and finds overlapping 45-minute gaps within working hours. | ~100-250 ms (in parallel) |
| 8 | Orchestrator | Both answers are back. It picks the best 3 slots (for example Tue 10:00, Wed 14:30, Thu 11:00) and builds a short reply from a template. | ~5 ms |
| 9 | Client App | Shows the three options to the recruiter. Total time so far: well under 1 second. | Total ≈ 600-850 ms |
| 10 | PostgreSQL + Monitoring | After the reply is sent, the request, result and timing are saved and the trace is recorded. The user does not wait for this. | async |

**Why it is fast:** steps 7a and 7b happen together, so the total is roughly the AI step plus the slower of the two lookups, not all of them added together. Cached data skips most of the Greenhouse wait.

## 4.2 Part B: Booking the chosen time

| # | Who | What happens, in simple terms | Time |
|---|---|---|---|
| 11 | User | Clicks "Wed 14:30". | 0 ms |
| 12 | Gateway → Orchestrator | Same checks as before. The request carries an idempotency key so a double-click cannot book twice. | ~15 ms |
| 13 | Calendar Service | Creates the meeting with Priya, Alex and Sam, with a video link. Returns the event ID. | ~200-300 ms |
| 14 | Orchestrator → Task Queue | Puts jobs on the queue: "record interview in Greenhouse", "send confirmation emails", "generate interview kit". | ~2 ms |

| # | Who | What happens, in simple terms | Time |
|---|---|---|---|
| 15 | Client App | Shows "Booked! Invites sent" to the recruiter. | Total ≈ 300-400 ms |
| 16 | Background Workers | Pick up the jobs. Write the scheduled interview to Greenhouse (with the On-Behalf-Of user ID), send emails, and ask the LLM to draft an interview kit. Each job retries on failure. | seconds, in background |
| 17 | Greenhouse webhook | Greenhouse notifies the ATS Service that the application changed, and the related cache entries are refreshed. | background |

**Why it is fast:** the user only waits for the calendar booking. Writing to Greenhouse, sending emails and generating the interview kit all happen in the background.

## 4.3 Shortcut: when the answer is already cached

1. Steps 1 to 4 are the same.
2. At step 5, Redis already has a fresh answer (a "cache hit").
3. The Orchestrator returns it straight away, skipping the AI, Greenhouse and calendar calls.
4. Total time is often under 100 ms.

## 4.4 What happens when something goes wrong

| Problem | What the system does | What the user sees |
|---|---|---|
| AI provider is slow | The call hits its timeout; the LLM Service retries once or switches to a backup model. | A short delay, or a simple form asking for the missing details. |
| Greenhouse rate limit reached | ATS Service reads the rate-limit headers, waits briefly and retries; uses cached data where possible. | Normally nothing; at worst a slightly slower reply. |
| Calendar token expired | Calendar Service refreshes the OAuth token automatically and retries. | Nothing. |
| A background job fails | Worker retries with growing waits (1 s, 2 s, 4 s...). After several failures the job goes to a dead-letter queue and an alert is raised. | Status shows "pending" until fixed. |

| Problem | What the system does | What the user sees |
|---|---|---|
| User double-clicks Book | The idempotency key is recognised; the second request returns the first result. | Only one meeting is booked. |
| One Orchestrator copy crashes | Load balancer health checks stop sending it traffic; other copies take over. | Nothing. |
| Invalid request | Pydantic validation rejects it with a clear message. | A friendly error explaining what is missing. |

## 4.5 Background flow: keeping data fresh

1. Something changes in Greenhouse, for example a candidate is moved to the next stage.
2. Greenhouse sends a webhook (an automatic message) to the ATS Service.
3. The ATS Service checks the webhook's signature to make sure it really came from Greenhouse.
4. It removes the old cached data for that candidate from Redis.
5. The next request fetches fresh data and caches it again.

# 5. How "Sub-Second" Was Achieved

## 5.1 Time budget for one request

| Stage | Typical time | How it was kept small |
|---|---|---|
| Network + gateway | 20-40 ms | Keep-alive connections; checks are lightweight. |
| Validation + auth context | ~5 ms | Pydantic models; permissions cached. |
| Cache lookup | ~1 ms | Redis in the same network region. |
| LLM understanding | 300-450 ms | Small fast model, short prompt, JSON schema, streaming, prompt caching. |
| ATS + Calendar (parallel) | 150-300 ms | Run at the same time; cached reference data; one batched free/busy call. |
| Merge + reply | ~5-10 ms | Plain Python, no extra calls; template replies. |
| Saving + logging | 0 ms for the user | Done after the reply is sent. |
| **Total (p95)** | **≈ 600-850 ms** | **Under the 1-second target.** |

## 5.2 The main speed techniques

- **Parallel calls (async):** independent calls run at the same time with asyncio.gather.
- **Caching:** frequently used data and repeated AI answers come from Redis in about 1 ms.
- **Background jobs:** anything the user does not need right now goes to the queue.
- **Right-sized AI:** a fast model for understanding requests; bigger models only where needed.
- **Streaming:** users see the reply start in a few hundred milliseconds.
- **Connection pooling:** reuse open connections to the database and external APIs instead of opening new ones every time.
- **Strict timeouts:** no single slow dependency can hold a request hostage.
- **Measure everything:** traces show exactly which step is slow, so effort goes where it matters.

## 5.3 Handling 1,000+ daily users

- Services are **stateless**, so more copies can be added at busy times (horizontal scaling, for example with Kubernetes or ECS autoscaling).
- The **load balancer** spreads requests evenly across copies.
- **Rate limiting** protects the system and stays within Greenhouse's and the AI provider's limits.
- **Workers scale separately** from the live API, so background bursts never slow users down.
- Each service is packaged in a **Docker container** and deployed independently.

## 5.4 Security and privacy

- All traffic is encrypted with HTTPS / TLS.
- Every request is authenticated (JWT) and checked against the user's permissions.
- API keys and OAuth secrets live in a secrets manager, never in code.
- Calendar tokens are stored encrypted in the database.
- Only the minimum candidate data needed is sent to the AI provider; personal details are masked where possible.
- Webhook signatures are verified; every change is logged with who made it (audit trail).

# 6. Explaining It in an Interview

## 6.1 The 30-second pitch

> **Say it like this**
>
> "I designed the backend for an AI recruiting assistant used by over a thousand people a day. A recruiter types a request in plain English; a FastAPI orchestrator uses an LLM to understand it, then calls separate services for Greenhouse and Google/Outlook calendars in parallel, and returns available slots in under a second at p95. I kept it fast with async fan-out, Redis caching, moving slow writes to a queue, and strict timeouts, and I kept it maintainable by giving every service a clear, versioned API contract."

## 6.2 Likely follow-up questions

| Question | Short answer |
|---|---|
| Why separate services instead of one app? | Each outside system has different speed, limits and failure modes. Isolating them means one slow or changing provider does not affect the others, and each can scale on its own. |
| How do you get sub-second with an LLM in the path? | Fast model for intent, short prompts, structured output, streaming, caching, and running the other lookups in parallel. |
| How do you handle Greenhouse rate limits? | Read the rate-limit headers, cache reference data, batch reads, retry with backoff, and push writes to background workers. |
| How do you avoid double-booking? | Idempotency keys on the booking request, stored in PostgreSQL. |
| How do you know it is fast? | p95 latency metrics and distributed traces per request, with alerts if p95 goes over one second. |
| What would you improve next? | Smarter slot ranking, semantic caching of AI answers, and load testing for 10x traffic. |